

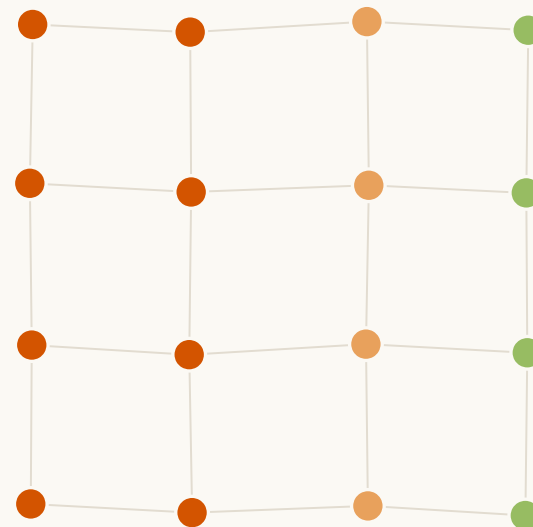
제24회 임베디드SW경진대회 · 자유공모 부문

# 불사

---

팀 화중생련 火中生蓮

고장(노드 파괴)을 데이터로 바꾸는 자가치유 산불 감시 메시 네트워크



# 재난에서 센서는 반드시 죽는다 — 우리는 그 죽음을 버리지 않는다

산불·지진·붕괴 현장에서 센서 노드는 파괴된다. 기존 시스템은 그 죽음을 손실로 처리하고 정보를 버린다. 불사는 죽은 노드의 시각과 좌표에서 불의 방향·속도·도달시각을 읽는다.

|        |                                                                                                           |
|--------|-----------------------------------------------------------------------------------------------------------|
| 작품명    | 불사(不死) — 고장을 데이터로 바꾸는 자가치유 산불 감시 메시                                                                       |
| 개발 배경  | 재난 현장에서 센서 노드의 파괴는 예외가 아니라 필연이다. 기존 메시는 죽은 노드를 우회할 뿐, 그 죽음에 담긴 정보를 버린다.                                   |
| 동기     | 대형 산불 대응 현장에서 지상의 실시간 화선 정보가 없어 판단이 늦어지는 국면을 보았다. 위성은 광역 개요만, 드론은 상공·간헐적이라 연기와 수목 아래 지표 화선의 초 단위 변화를 놓친다. |
| 목표     | ① 교차검증된 '죽음'을 확정 신호로 모아, 죽은 노드의 좌표와 시각에서 화선의 방향·속도·도달시각을 추정한다.<br>② 경로가 끊겨도 남은 노드가 우회로를 다시 짜 관측을 지속한다.    |
| 필요성    | 지상에서 화선을 실측하는 저가 레이어가 없다. 노드 1대 부품비가 1.2만 원(데모 구성 기준)이라, 타 죽는 것을 전제로 촘촘히 깔 수 있다.                          |
| 소스코드   | (URL 확보 후 삽입)                                                                                             |
| 시연 동영상 | (URL 확보 후 삽입)                                                                                             |

독창성은 응용(산불)이 아니라 아키텍처에 있다 — 고장을 데이터로, 그리고 자가치유로.



# 상공은 지표 화선을 못 본다 — 그 공백이 판단을 늦춘다

## 무엇이 안 보이나

연기와 수목 캐노피 아래에서 실제로 번지는 지표 화선(fire front)의 위치·방향·속도. 위성은 광역 개요만, 드론은 상공·간헐적이라 초 단위 국지 변화를 놓친다.

## 왜 치명적인가

진화 지휘부의 자원 배치와 최전선 대원의 대피 타이밍이 바로 이 정보에 달려 있다. 공백이 곧 인명·재산 피해로 이어진다.

## 왜 지상 관측 레이어가 없나

- 위성은 재방문 주기가 있어 초 단위 변화를 못 본다.
  - 드론은 체공 시간이 제한되고 야간·질은 연기에서 관측이 어렵다.
  - 지상 고정 센서는 불에 타면 끝이라, 화선이 지나가는 구간에는 배치하지 않는다.
- 정작 화선이 지나가는 그 자리에 관측이 비어 있다.



# 죽은 시각이 곧 불이 도착한 시각이다

## ① 노드 파괴

불에 타 죽는 순간

## ② 죽은 시각·좌표

= 불이 도착한 시각

## ③ 방향·속도·ETA

+ 신뢰도 등급

### 목표 1 — 고장의 데이터화

교차검증된 '죽음'을 확정 이진 신호로 모아, 흩어진 좌표와 죽은 시각에서 화선의 위치·방향·속도를 추정한다.

### 목표 2 — 자가치유

노드가 죽어 경로가 끊겨도 남은 노드들이 자동으로 우회로를 다시 짜, 관측 자체가 계속되게 한다.

→ 두 목표가 합쳐져: 노드는 죽지만(불에 타도) 시스템의 관측은 '죽지 않는다'. = 불사(不死)의 이중 의미.



# ESP32 17대(노드 16 + 브리지 1)와 라즈베리파이 1대로 이뤄진다

## 하드웨어

- 센서 노드: ESP32 × 16 (WiFi 메시)
- 메시 브리지: ESP32 × 1 (id 99) — 메시↔USB 시리얼 중계
- 온도 감지: DS18B20 디지털 온도센서 (9비트 고속 모드)
- 상태 표시: WS2812 LED (생사 시각화)
- 게이트웨이: 라즈베리파이 4 — 시리얼 수신 · 추정 · 대시보드
- 전원: USB 5V 급전 (데모) — 배터리 급전은 향후 과제

## 소프트웨어

- 노드 펌웨어: C/C++ (Arduino-ESP32 + painlessMesh)
  - 메시 자동 구성·우회 + 메시 시각 동기 제공
  - 브리지도 같은 펌웨어를 쓴다(인덱스만 99) — 단일 코드베이스
- 게이트웨이: Python — 시물 추정기 코드 그대로 재사용
- 관제: 자체완결 웹 대시보드 (HTML, 서버 불필요)
- 개발·검증: 네트워크 시뮬레이터(Python)
- 판정 일원화: 화재/비화재 판별을 단일 함수로 — 시물과 실물이 같은 코드를 호출한다(시물 결과가 실물의 근거가 되도록).

### 센서 노드

ESP32 + 온도

### 메시 네트워크

painlessMesh

### 게이트웨이

RPi · 추정기(Python)

### 웹 대시보드

화선·경보 관제



## 네 모듈이 죽음을 데이터로 바꾼다

①

### 임종신호 (Last-Gasp)

노드가 임계온도를 넘겨 죽기 직전 쓰는 마지막 패킷 — 죽음의 시각을 확정.

②

### 자가치유 라우팅

노드가 죽어 길이 끊기면 sink 기준 역방향 BFS로 우회로를 자동 재계산.

③

### 오탐 방어 (교차검증) ★

통신두절 vs 진짜 파괴를 다중이웃 + 온도로 구분 — 순정 painlessMesh가 못 하는 계층.

④

### 도착시각장(場) 추정 ★

죽은 좌표와 시각에서 최소제곱으로 방향·속도·ETA와 신뢰도를 역산.



# 죽은 점들에 면을 씌우면 그 기울기가 불의 방향이다

각 노드가 타 죽은 시각 = 불이 그 지점에 도착한 시각. 흩어진 (x, y, 죽은시각)에 면을 맞추면 그 기울기가 곧 화선이다.

실제 구현은 **2단 구조** — 노드마다 국소 적합을 돌리고, 그 결과들을 신뢰도로 가중해 합친다.

지형도에서 등고선이 촘촘한 쪽이 급경사인 것과 같다 — 도착시각의 등고선이 화선이고, 가장 빨리 변하는 방향이 불이 가는 방향이다.

## 1단 · 국소 평면 적합

노드 j마다 독립적으로:

- 무선 이웃  $n$  창 안의 사망들을 모아
- 최소제곱으로 평면을 맞추고
- 기울기  $\nabla T_j$  를 얻는다

→ 노드 수만큼 N개의 추정치

## 2단 · 집계

N개를 하나로 합친다:

- 방향 — 신뢰도 가중 벡터합
- 속도 — 중앙값(강건 통계)
- 진단 — 유효표본수  $n_{\text{eff}}$
- 두 출력이 다른 집계를 쓴다

## 출력

- 진행 방향  $\hat{n}$
- 진행 속도  $s$
- 지점별 도착예상시각 ETA
- 신뢰도 등급

정보가 부족하면 값을 내지 않고 INSUFFICIENT를 반환한다.

$$\nabla T = \hat{n} / s \rightarrow \text{방향 } \hat{n} = \nabla T / |\nabla T|, \text{ 속도 } s = 1/|\nabla T|$$

학습 데이터 불필요 · 라즈베리파이에서 즉시 계산

모든 노드에 똑같이 걸리는 센서 지연은 평면의 절편에만 흡수되어 기울기에서 상쇄된다 — **방향·속도가 센서 지연에 원리적으로 면역인 이유(수식 증명).**



# 통신두절과 진짜 파괴를 가르는 판정

노드가 조용해졌다고 다 죽은 게 아니다. 잠깐 끊긴 것(통신두절)이나 불이 아닌 죽음(배터리·고장)을 화재로 오판하면 가짜 화선이 그려진다.



임종신호는 있으면 증거이고, 없어도 기각 근거가 아니다 — 전소로 신호조차 못 보낸 노드가 바로 진짜 화재이기 때문이다.

판별은 말단이 아니라 이웃 정보가 모두 모이는 게이트웨이에서 한다. · [시물] 오탐률 0% · 비화재 COOL 누수 86.4% → 0%





## 노드가 죽어도 경로는 살아난다 — 15.9초 만에 재구성

### 자가치유 라우팅

- 브리지(id 99) 기준 역방향 BFS로 각 노드의 상행 경로를 구성.
- 노드가 DEAD로 확정되면 영향 구역만 경로를 즉시 재계산.
- 연결성이 유지되는 한, 죽은 노드를 우회해 데이터가 계속 sink에 도달.

메시는 노드들을 나무처럼 잇는다. 가지 중간의 노드가 죽으면 그 아래 노드들은 게이트웨이로 가는 길을 잃는다.

- 실물 메시 실측 (16대 중 15대 합류 — 1대는 이 관측 창에서 메시에 나타나지 않았다 · 2026-09-01)

하위 11대를 거느린 중계 노드 1대를 물리 제거 → 재부팅 없이 15.9초 만에 경로 재구성.

9개 노드가 부모를 바꿨고, 남은 14대가 70~80초 내 전송 재개.

### 임종 신호 (Last-Gasp)

- 노드가 임계온도를 넘어 죽기 직전, 마지막 패킷을 방출.
- '언제 죽었는가'를 확정 → 도착시각 추정의 입력 시각을 정밀화.
- 하트비트(주기 신호) 누락과 결합해 침묵을 판정.

죽음의 '시각'이 정확할수록 화선 방향·속도 추정이 정밀해진다.

자가치유(관측 지속) + 임종 신호(죽음의 시각 확정)가 만나, '죽는 노드'가 '살아있는 정보'가 된다.



# 결과가 나빠도 파라미터를 고치지 않는다

## 저장소 구성

- sim/ — 시뮬레이터 · 추정기(estimator) · 집계(aggregate)
- firmware/ — ESP32 노드 펌웨어(C/C++)
- gateway/ — 라즈베리파이 추정기(Python, sim 재사용)
- dashboard/ — 자체완결 웹 관제
- docs/ — 연구노트(PROGRESS · DECISIONS · 알고리즘 명세)

결정 로그(D-001~)를 남겨 '왜 그렇게 정했는가'를 전부 추적 가능하게.

- 게이트웨이가 시뮬레이터의 추정기 코드를 그대로 재사용한다.
- 화재/비화재 판별도 단일 함수 — 시물과 실물이 같은 판정을 쓴다.

## 추정기 동결 + 버전 분기

- 핵심 추정기 파일을 동결(frozen) — 모든 개선은 버전 분기로만.
- 그래서 어떤 실험도 과거 결과를 훼손할 수 없다.  
회귀가 매번 비트 단위 동일함을 확인(최대차 0.000e+00).
- 자동 테스트 54건 상시 통과.
- 결과가 나빠도 파라미터를 유리하게 고치지 않는다(기록만).  
→ 데모에만 맞춘 과적합을 피한다.
- 버그는 고친다. 파라미터는 만지지 않는다 — 둘을 구분한다.
- 예측을 미리 등록하고 맞든 틀리든 기록한다. 기각된 예측 5건이 문서에 남아 있다.

→ 값을 정하기 전에 먼저 재고, 잔 값에서 파라미터를 유도한다. 완성도는 '보기 좋은 숫자'가 아니라 '재현 가능하고 정직하게 측정된 숫자'에서 나온다.



# 가중치의 부호 하나가 방향을 무너뜨리고 있었다 — 82° → 0.03°

타원 화선의 측면 조건에서 방향오차가 82~86°. 90°가 무작위 추측의 한계이므로 사실상 아무 정보도 못 주는 상태였다. 세 번의 진단이 전부 측정으로 기각된 뒤에야 원인에 닿았다.

|   | 가설                         | 검정 방법                 | 결과                       |
|---|----------------------------|-----------------------|--------------------------|
| ① | 곡률 탓 — 화선이 굽어 평면 근사가 깨짐    | 창을 넓혀 곡률 영향을 분리       | 기각 — 창을 넓히자 81.8° → 5.6° |
| ② | 채점 방식 탓 — 국소 법선 vs 전역 축 혼동 | 채점 코드를 직접 확인          | 기각 — 이미 국소 법선 기준이었다      |
| ③ | 표본 부족 탓 — 창 안에 이웃이 모자람     | 표본 수·조건수 진단           | 부분 기각 — 13개인데도 실패        |
| ④ | 집계 가중치가 원인                 | 노드별 개별 오차와 가중치 분포를 분해 | ★ 확정                     |

+0.986

corr(속도, 방향오차)

속도가 큰 노드가 곧 틀린 노드

86.4%

한 노드의 가중치 비중

그 노드의 개별 오차 85.21°

1.35°

개별 오차의 중앙값

나머지 노드들은 맞히고 있었다

정답은 표본 안에 있었다 — 집계가 그것을 버리고 최악의 노드 하나를 골랐다.  $w = 1/|VT|$  를  $w = |VT|^2$  로 교정. 파라미터를 유리하게 조정한 것이 아니다. 오차 전파식에서 다시 유도하니 부호가 반대였다.



# 값을 내지 않는 것도 기능이다 — 다만 그게 진짜인지 검증했다

정보가 없으면 값을 내지 않는다 — 우리는 이것 강점으로 여겼다.  
그런데 실물로 옮기자 16노드가 전부 값을 내지 못했다. 기능이 작동한 것인지, 우리가 잘못 정한 값 때문인지 구분해야 했다.

## 설계 결정 — 폴백을 넣지 않는다

- 산출 대응에서 「모른다」와 「90도 틀린 확신」의 비용은 대칭이 아니다. 방향을 모르면 판단을 보류하지만, 틀린 방향을 믿으면 대원을 화선 쪽으로 보낸다.
- 그래서 표본이 모자라면 값을 내지 않고 INSUFFICIENT 를 반환하게 만들었다. 로그에는 "냈다면 틀렸을 것: True" 가 함께 남는다.

## 진단 — 정보가 없었나, 창이 좁았나

- 창(window)이란 「얼마나 오래된 죽음까지 한 계산에 넣을 것인가」다. 창이 좁으면 이웃의 죽음이 창에 못 들어와, 노드가 자기 하나만 보고 계산하게 된다.
- 창을 넓혀가며 다시 재니 산출 성공이 2 → 12 → 15 → 16 으로 올랐다. 정보가 없었던 게 아니라 창이 좁았다.

창 크기 스윙 — 8/30 드라이런 조건(화선 1.1 mm/s)에서 켜 값 [당시 값]

| 창 크기 (초) | 산출 성공   | 전역 방향  | 참값 55.48° 대비 |
|----------|---------|--------|--------------|
| 60       | 2 / 16  | 30.94° | -24.54°      |
| 120      | 12 / 16 | 42.17° | -13.31°      |
| 173      | 15 / 16 | 56.26° | +0.78°       |
| 240      | 16 / 16 | 55.76° | +0.27°       |
| 327      | 16 / 16 | 55.83° | +0.35°       |

값을 새로 고르는 대신 구조를 바꿨다 — 창 크기를 물리에서 유도하게 했다. 창 = 통신반경 ÷ 화선속도.  
이 식이 622.3초를 내놓았고, 그 값으로 본편 런을 돌려 증거가 확보된 15건의 죽음에서 방향 오차 2.85도를 냈다.

지금은 진짜로 정보가 부족한 조건에서만 값을 내지 않는다. 유효표본수  $n_{eff}$  로 「표본이 없는 것」과 「한 노드가 결과를 독식한 것」을 구분한다.



# 실물에서만 터지는 위험 셋을 터지기 전에 잡았다

## ① 시계 동기

서로 다른 노드의 사망 시각을 빼서  
기울기를 만드는 알고리즘이므로  
시간축이 같아야 한다.

초기 펌웨어는 보드 부팅 기준 시계를  
썼다 — 보드마다 원점이 달라 뺄셈  
자체가 무의미해지는 원리적 파탄.

→ 메시 동기 시각으로 교체. 사망 시각도  
'루트의 확정 시각'에서 '당사자 노드가  
각인한 시각'으로 (확정 시각엔 감지·투표  
지연이 섞여 있었다).

## ② 센서 열관성 $\tau$

센서는 공기 온도를 즉시 따라가지 못한다.  
이 지연이 곧 사망 시각의 오차가 된다.

시물 시험 범위 0 ~ 10초  
실측 범위 11 ~ 92초

→ 검증했다고 믿은 범위 밖에 실재가  
있었다.

★  $\tau$ 는 온도가 아니라 유속의 함수 — 같은  
센서·같은 시행 안에서 제트 21.5초 / 실내  
92초로 4.1배 차이. 바뀐 것은 공기가  
움직이느냐뿐이었다.

## ③ 실물 파이프라인 공백

시물에는 화재/비화재 판별이 있는데  
실물 경로에는 없었다 — 게이트웨이가  
판별기를 아예 호출하지 않고 있었다.

→ 판정 로직을 한 함수로 통합해 시물과  
실물이 같은 코드를 호출하도록 교정.

이걸 놓쳤으면 시물에서 검증한 방어가  
실물에서 통째로 빠진 채 데모를 했을  
것이다.

세 위험 모두 **실물에서 터지기 전에** 발견했다. ①은 코드 검토로, ②는 실측으로, ③은 실물 이식 준비 중에 — 각각 다른 방법이 필요했다는 것이 이 문제들의 성격을 보여준다.



# 감지는 그들이, 추적은 우리가

| 비교 대상          | 그들의 방식                    | 한계                           | 우리 (불사)                             |
|----------------|---------------------------|------------------------------|-------------------------------------|
| 온도 스트리밍        | 살아있는 노드가 온도 상시 전송         | 초저가 대량 노드엔 상시 전송 부담 (전력·대역폭) | 교차검증된 '죽음'을 확정 이진 신호로 → 화선 위치·방향·속도 |
| Dryad Silvanet | 불 오기 전 조기 감지(내열, +85℃ 한계) | 불 속에선 못 버팀 — 감지 특화, 화선 추적 아님 | 타 죽으며 화선을 그림 — 감지(그들)와 추적(우리)은 상보   |
| 순정 ESP-MESH    | 노드 죽으면 경로 우회(연결성만)        | 죽음을 손실로만 처리, 정보로 안 씀         | 죽음을 1급 이벤트로 승격 + 도착시각 추정 + 오탐 방어    |
| 드론·위성          | 상공·우주 광역 개요               | 연기·수목 아래 지표 화선·초 단위 변화 못 봄   | 지표 실측 레이어 + 지상 통신 백본 — 대체 아니라 상보    |

★ 헤드라인 차별점    오탐 방어(통신두절 vs 진짜 파괴 구분) = 순정 painlessMesh가 못 하는 우리만의 계층.



## 지휘부는 배치를, 대원은 대피 시각을 얻는다

### 수요자 1 · 진화 지휘부

지표 화선의 실시간 위치·방향·속도로 '어디에 자원을 먼저 투입할지' 판단. 자원 배치 정확도 ↑.

### 수요자 2 · 최전선 대원

'이 구역에 화선 N분 내 도달' 대피 타이밍 제공. 대원 안전과 직결 (속도는 안전마진으로 보수적 경보).

### 배포 · 고위험 구역 사전 배치

상습 발화지, 도시·마을 인접 산림(WUI), 주요 시설·문화재·등산로에 핀포인트. 초저가라 촘촘히 깔아도 부담 적음(사전 배치는 상용 선례로 검증).

### 기대효과

상공 관측이 놓치는 지표 화선을 지상에서 실시간화 → 자원 배치 정확도와 대원 안전을 동시에 끌어올린다.

# 타 죽는 것을 전제로 하면 경제 논리가 뒤집힌다

## 비용 논리

- 불 속에 들어가는 역할엔 내구성을 '살' 수 없다 → 저가 소모형이 유일하게 합리적.
- 불날 때마다 갈아끼우는 모델이 오히려 경제적.
- 노드 1대 부품비 1.2만 원 [데모 기준 — 케이스·야외전원·거치 제외].
- 데모 조달 총액 ≈ 45만 원 (예비 보드·공구·라즈베리파이 포함, 부가세 포함).
- 예방 투자 vs 산불 1회 피해(인명·산림·재산) = 후자가 압도적.

## 발전 가능성

- 같은 '고장-패턴-데이터화 + 자가치유' 구조를 실내 화재·가스 누출·구조물 붕괴·홍수로 확장 가능.
- 온도 상승 궤적·곡선·허용 시공간 정합·다중소스 추정(#2e)이 다음 연구 축.
- 드론·위성과 대체가 아니라 상보 레이어로 통합.
- 전력: 상시 메시 구조상 노드당 90~130 mA [추정 — 데이터시트 기준]. 저전력 모드(평시 deep sleep · 화재 시 메시 기동)는 향후 과제.

→ 독창성은 특정 응용이 아니라 '고장을 데이터로 승격하는 아키텍처'에 있어, 재난 전반으로 이식된다.



# 1인이 네 영역을 병렬로 — 규율과 도구로 범위를 통제했다

| 단계         | 기간        | 내용                          |
|------------|-----------|-----------------------------|
| ① 시뮬·알고리즘  | 7월        | 시뮬레이터 · 도착시각 추정 · 스트레스 시험   |
| ② 알고리즘 정밀화 | 8월 초      | 집계 결함 교정 · 자기 차단 도입 · 명세 대조 |
| ③ 실측·실물 포팅 | 8월 중      | 센서 열관성 실측 · 펌웨어 포팅 · 메시 실증  |
| ④ 자가치유 실증  | 8월 말      | 4노드 메시 · 중계 이탈/복귀 실측        |
| ⑤ 조립·교정    | 8월 말      | 16노드 조립 · 감지부 좌표 · 열원 교정    |
| ⑥ 리허설·촬영   | 8월 말~9월 초 | 16노드 본편 촬영 · 자가치유 실증        |
| ⑦ 문서·제출    | 9월 초      | 영상 편집 · 보고서 완성 · GitHub 공개  |

## 1인 업무 분장 (영역별)

펌웨어(ESP32·painlessMesh) · 알고리즘/게이트웨이(Python 추정기) · 하드웨어(16노드 조립·데모·계측) · 문서/발표(보고서·영상)를 단독으로 단계적 수행. 시뮬 선개발로 리스크를 앞당겨 해소하고, 추정기 동결·사전등록 규율로 범위를 통제 — 1인 개발의 범위 관리·완결 역량을 보인다. 설계·판단·실험·검증은 단독 수행. 반복 구현과 문서 정리에 AI 코딩 보조(Claude)를 활용.

